Exploit Development Basics

Common Exploit Types

1. Buffer Overflow

   A buffer overflow is a vulnerability that occurs when a program writes more data into a memory buffer than it can handle, causing adjacent memory to be overwritten. This can lead to application crashes or allow attackers to execute arbitrary code on the system. It is commonly found in low-level languages like C and C++ where memory management is manual.

2. SQL Injection (SQLi)

   SQL Injection is a web vulnerability that arises when user input is not properly validated and is directly included in SQL queries. Attackers can manipulate the query to bypass authentication, retrieve sensitive data, or modify the database. It is one of the most critical vulnerabilities affecting database-driven applications.

   Lab Setup
   Attacker: Kali Linux
   Target: http://testfire.net/

   Perform SQL Injection Attack

   - Payload:
     ' OR 1=1 –

   - Enter in login:

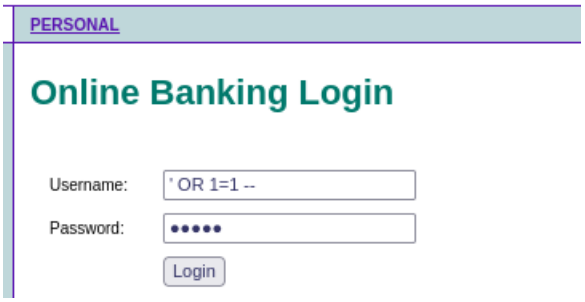

Figure 1: Enter payload in username

- Password:
  'anything'

- Result

  Login successful **without valid credentials**

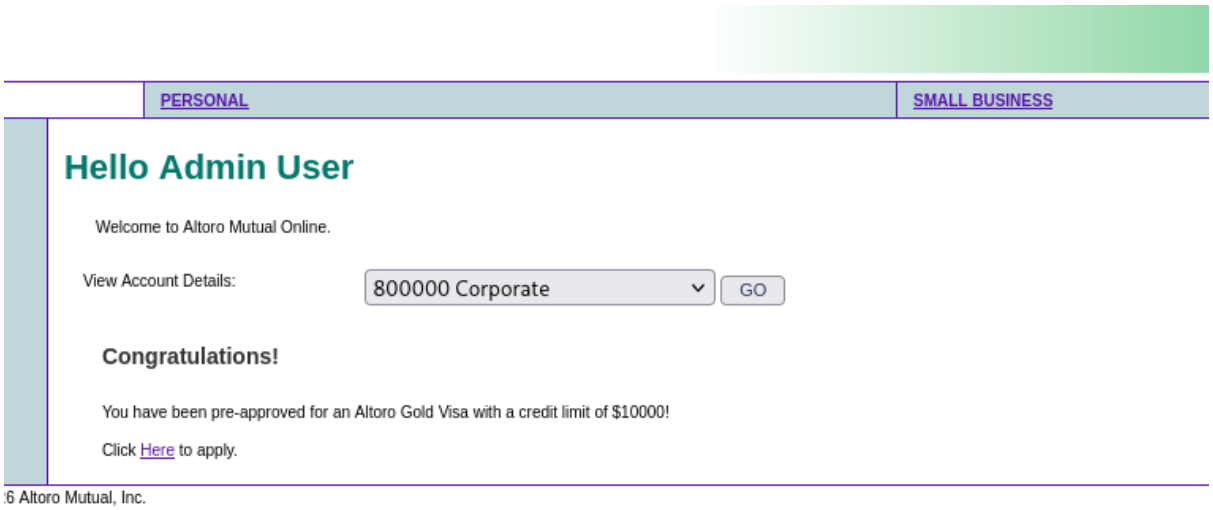

Figure 2: Login was successful, and the user was authenticated as admin

3. Cross-Site Scripting (XSS):

   Cross-Site Scripting (XSS) occurs when a web application fails to sanitize user input, allowing attackers to inject malicious scripts into web pages viewed by other users. These scripts execute in the victim's browser and can steal cookies, hijack sessions, or deface websites. It is commonly exploited through unescaped input fields.

   Lab Setup
   Attacker: Kali Linux
   Target: http://testfire.net/

   - Identify Input Field
     Comment box / search box / input form

   - Inject XSS Payload
     <script>alert('XSS')</script>

Figure 3: Inject XSS Payload

- Result
  A popup alert appears in the browser

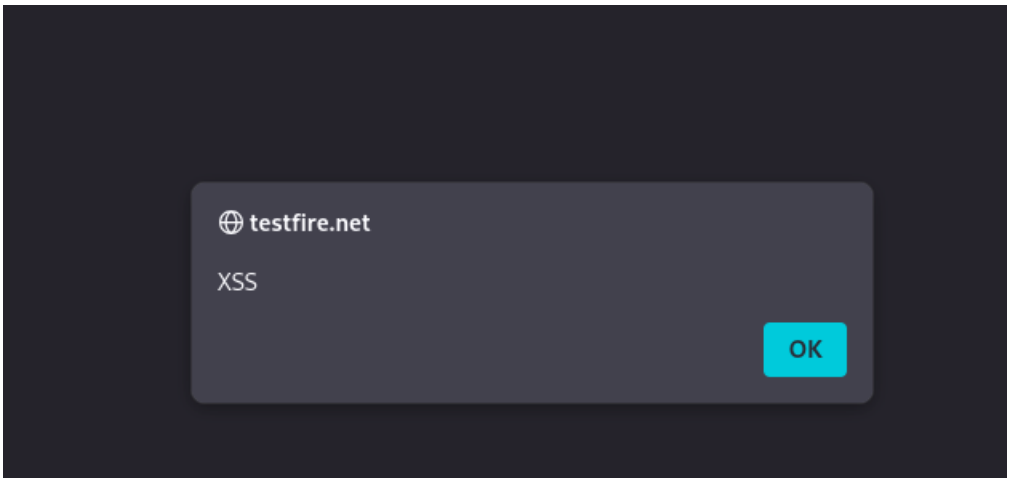

Figure 4: JavaScript is executed

Exploit Writing

Tools Used

- Python
- Exploit-DB
- GDB (for debugging)

1. Select a PoC from Exploit-DB
   - Search for a known vulnerability
   - Download the PoC exploit code
   - Study how it works (input size, payload, target)

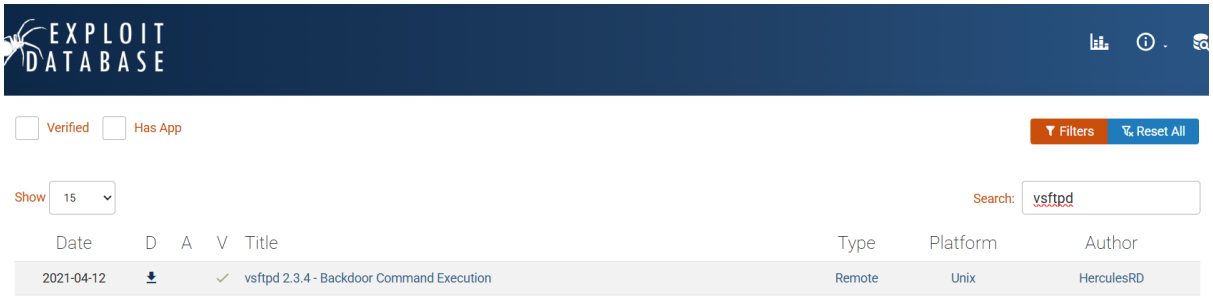

Figure 5: Search for vsftp vulnerability on Exploit-DB

2. Understand the Exploit Structure
   Basic exploit components:
   - Buffer (padding) → fills memory
   - Return address (EIP/RIP overwrite)
   - Payload (shellcode)

3. Simple Python Exploit Example

```
import socket

target_ip = "192.168.1.10"
port = 9999

# Step 1: Create buffer
buffer = "A" * 100

# Step 2: Overwrite return address (example)
eip = "B" * 4

# Step 3: Combine payload
payload = buffer + eip

# Step 4: Send exploit
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((target_ip, port))
s.send(payload.encode())
s.close()
```

4. Testing the Exploit
   - Run the vulnerable program
   - Execute the Python script
   - Observe crash or control over execution

```
┌──(kali㉿kali)-[~]
└─$ nano python_exploit.py

┌──(kali㉿kali)-[~]
└─$ python3 python_exploit.py
Traceback (most recent call last):
  File "/home/kali/python_exploit.py", line 17, in <module>
    s.connect((target_ip, port))
    ~~~~~~~~~^^^^^^^^^^^^^^^^^^^^
ConnectionRefusedError: [Errno 111] Connection refused
```

Figure 6: Testing the Python Exploit

5. Expected Outcome
   - Application crash or abnormal behavior
   - Potential control over instruction pointer
   - Foundation for advanced exploitation

Mitigations

1. ASLR (Address Space Layout Randomization)

   ASLR is a security technique that randomizes memory locations used by system processes, such as stack, heap, and libraries. This makes it difficult for attackers to predict addresses required for exploits like buffer overflows, thereby reducing the chances of successful code execution.

2. WAF (Web Application Firewall)

   A WAF is a security layer that filters and monitors HTTP traffic between users and web applications. It detects and blocks malicious requests such as SQL Injection and XSS attacks by applying predefined rules and patterns, helping protect web applications from common threats.

3. Patching

Patching involves regularly updating software to fix known vulnerabilities. Vendors release patches to address security flaws, and applying them promptly reduces the risk of exploitation. Unpatched systems are one of the most common entry points for attackers.

Exploit-DB for PoC Examples

Exploit-DB is a public database that contains a large collection of Proof of Concept (PoC) exploits for known vulnerabilities. By studying these examples, learners can understand how real-world vulnerabilities are exploited and how exploit code is structured. It helps in analysing attack techniques, payload construction, and target behaviour. Reviewing PoCs also improves practical skills in penetration testing and vulnerability assessment.

Use TCM Security's Exploit Guides

TCM Security provides practical and beginner-friendly exploit development guides that help learners understand real-world attack techniques. These guides explain concepts such as buffer overflows, privilege escalation, and web application attacks in a step-by-step manner. By following these resources, students can gain hands-on experience in identifying vulnerabilities and developing exploits in a controlled lab environment.

Try TryHackMe's Buffer Overflow room

Sudo Buffer Overflow

The Sudo Buffer Overflow room on TryHackMe teaches how a vulnerability in the Linux sudo program can allow a normal user to gain root (administrator) privileges. It demonstrates the vulnerability CVE-2021-3156, where improper handling of input causes a heap buffer overflow in sudo. In the lab, learners check the installed sudo version, identify that it is vulnerable, find a public exploit, and run it to escalate privileges from a normal user to root. This room helps students understand Linux privilege escalation and real-world security vulnerabilities

```
┌──(kali㉿kali)-[~]
└─$ ssh -p 4444 tryhackme@10.67.162.10
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
tryhackme@10.67.162.10's password:
Last login: Sat Feb  8 05:00:59 2020 from 192.168.1.209
tryhackme@sudo-bof:~$ whoami
tryhackme
tryhackme@sudo-bof:~$ ls
exploit
tryhackme@sudo-bof:~$ ./exploit
[sudo] password for tryhackme:
Sorry, try again.
# tryhackme
sh: 1: tryhackme: not found
# whoami
root
```

*Figure 7: **Gaining root privileges** on the system.*

```
# cat /root/root.txt
THM{buff3r_0v3rfl0w_rul3s}
#
```

Figure 8: Flag in /root/root.txt

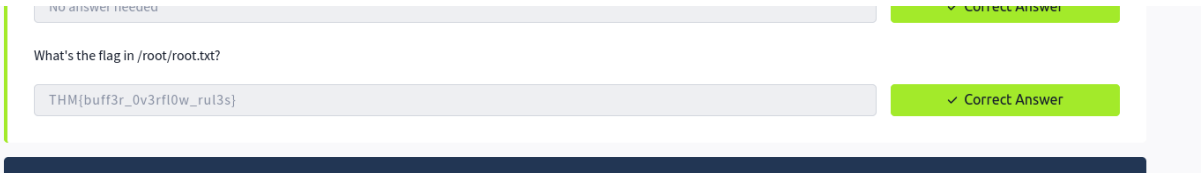

Figure 9: The lab was completed successfully

Conclusion

This lab demonstrated the basics of exploit development and common vulnerabilities such as buffer overflow, SQL injection, and cross-site scripting. By performing practical exercises using tools like Python, Exploit-DB, and Kali Linux, we learned how attackers exploit insecure applications and how vulnerabilities can lead to serious impacts such as gaining root privileges. The TryHackMe sudo buffer overflow lab helped understand Linux privilege escalation in a controlled environment. Overall, this exercise provided practical knowledge about identifying vulnerabilities, developing exploits, and understanding the importance of security mitigations.